# Algebraic World Model: From Exploration to Proofs
# Supplementary Material

**Anonymous authors**
Paper under double-blind review

References to sections, figures, tables and equations without a letter refer to the main paper.

## A Formalization details

**A.1 Environment.** Following Zhao et al. (2026), an environment is a deterministic, goal-directed special case of a partially observable Markov decision process (Kaelbling et al., 1998): states $\mathcal{S}$, actions $\mathcal{A}$, observations $\mathcal{O}$, a transition function $T : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$, an observation function $\rho : \mathcal{S} \to \mathcal{O}$ and goal states $G \subseteq \mathcal{S}$. The agent sees $o_t = \rho(s_t)$ and receives a signal when a level is cleared. A transition is a triple $x = (o, a, o')$, and a game's transition corpus $\mathcal{D}$ is the set of distinct transitions across its runs, deduplicated by content hash.

**A.2 Concepts and the gate.** A concept $c = (\sigma_c, \alpha_c, \gamma_c, E_c)$ has an optional executable scope $\sigma_c$, an assertion $\alpha_c$ returning TRUE, FALSE or NA, an optional predictor $\gamma_c$, and a non-empty evidence set $E_c$. When the scope rejects $x$ the verdict $v_c(x)$ is NA, otherwise it is $\alpha_c(x)$; an exception, an invalid return value or a timeout gives ERR. A concept is scored only on transitions outside $E_c$; a concept with no TRUE or FALSE verdict there is reported as unmeasured, not blocked. The gate of Eq. (1) is enforced on the action command: it refuses the next action while the latest settlement has a FALSE or ERR verdict on any $x \in \mathcal{D}$, including the concept's own evidence.

**A.3 Objects, rules and generalization.** A partial parser $\pi : \widetilde{\mathcal{O}} \rightharpoonup \mathcal{W}$ maps an observation, augmented by any explicitly supplied history or auxiliary information, to a world of typed objects. Below, $\pi(o)$ abbreviates parsing with that context. Pixel interpretation is confined to the parser, and parse failures are reported rather than guessed. A rule $r = (\sigma_r, \alpha_r)$ has a scope and an assertion on $(\pi(o), a, \pi(o'))$. Each rule is scored on every recorded step by the number of agreeing and disagreeing steps; a step on which no rule gives a verdict is unclaimed. The elimination diagnostic set $\mathcal{N}$ of Eq. (2) contains at most 120 corrupted transitions per game, with a fixed seed; larger corpora used in some merge comparisons are identified separately in Table 3. In the elimination set, each replaces part of a recorded transition $(o, a, o')$ in one of five ways: the successor is $o$ itself (nothing happened); the successor of the same action on another transition; the predecessor of another transition; $o'$ with a few cells changed that do vary in the corpus; or another transition's $o$ (swapped precondition). We set

$$\text{Rej}(r) = \{m \in \mathcal{N} : v_r(m) = \mathsf{FALSE}\}, \tag{A.1}$$

and a rule $r$ is flagged as redundant on $\mathcal{N}$ when the other rules already reject what it rejects,

$$\text{Rej}(r) \subseteq \textstyle\bigcup_{r' \neq r} \text{Rej}(r'), \tag{A.2}$$

provided that the checks on the recorded transitions still pass. A field is accepted as redundant on the diagnostic corpus when erasing it to a constant changes no tested verdict on $\mathcal{D} \cup \mathcal{N}$; this does not establish redundancy on unobserved states.

**A.4 Lean statements.** The step function dispatches among the rule definitions, returning a successor or a terminal outcome for actions admitted by the model's legality conditions. The treatment of illegal inputs and terminal states is specified by each game's formal definitions. $\text{everWon}(w, \bar{a})$ holds when some legal prefix of $\bar{a}$ reaches a winning outcome. The two statements of §2.4 have the following form, with action sequences interpreted according to those game-specific legality conditions:

$$\text{Winnable}(w) :\Leftrightarrow \exists\, \bar{a}.\ \text{everWon}(w, \bar{a}) = \mathsf{true}, \qquad \text{Unsolvable}(w) :\Leftrightarrow \forall\, \bar{a}.\ \text{everWon}(w, \bar{a}) = \mathsf{false}.$$

For level $\ell$, the initial world $w_0^\ell$ is parsed from that level's first screen in the certified run (for some games, hidden fields read from another frame of the same level are supplied), and the certified actions are $\bar{a}^\ell$; the reachability proof (a replay of a recorded run; see §3 on what this measures) establishes traceOk$(w_0^\ell, \bar{a}^\ell)$, i.e. that every computed state agrees with the recorded frame on the components the formal model represents, and establishes Winnable$(w_0^\ell)$ with $\bar{a}^\ell$ as witness. Each theorem is closed by kernel reduction and its axioms lie within $\{$`propext`, `Classical.choice`, `Quot.sound`$\}$, with no `native_decide`.

The release check uses Lean 4.34.0 to compile the published `Game.lean` entry points of all 25 games and audit their `consistent` and `faithful` theorems; each compiles and both theorems depend only on `propext` (Table 1). The 1,374 theorem declarations in Table 2 describe the artifact inventory, not 1,374 independent rebuild trials. For `lf52` and `wa30` this recheck replaces the record made at sealing: there the local check failed with two kernel `decide` errors, so both theorems depended on `sorryAx`. The published files, with sha256 prefixes `888f93639f4854f8` (`lf52`) and `e367f8f2fab17834` (`wa30`), now compile with Lean 4.34.0 and print `'consistent' depends on axioms: [propext]` and `'faithful' depends on axioms: [propext]`.

Table 1: Recheck of the published `Game.lean` of every game with Lean 4.34.0 on 2026-10-24. Each file is compiled as published, followed by `#print axioms consistent` and `#print axioms faithful`, with `lean +leanprover/lean4:v4.34.0 -M60000 Check.lean`; every run returns 0 with no error. *sha256*: first 16 hex digits of the published file; *Axioms*: the union over the two theorems.

| Game | sha256 | Seconds | Peak GB | Axioms |
|---|---|---|---|---|
| ar25 | e7af25060c66dd11 | 78 | 5.4 | propext |
| bp35 | f455a788e565472f | 50 | 4.3 | propext |
| cd82 | 35d730265f95fe57 | 81 | 3.8 | propext |
| cn04 | 8f0ab9ec07495b3b | 74 | 3.3 | propext |
| dc22 | d1021cb4846d8463 | 426 | 18.6 | propext |
| ft09 | ce3b3d4eb6c119f9 | 57 | 5.5 | propext |
| g50t | d0aecf39aac07317 | 52 | 3.2 | propext |
| ka59 | a2016b3b6dc4f067 | 148 | 7.8 | propext |
| lf52 | 888f93639f4854f8 | 826 | 35.7 | propext |
| lp85 | d87130ce3c1de9d1 | 210 | 11.7 | propext |
| ls20 | 287c7dcf9ce693fc | 256 | 7.8 | propext |
| m0r0 | 3fccb87cba48feed | 45 | 1.7 | propext |
| r11l | 9ce832789da3054c | 252 | 16.8 | propext |
| re86 | 0f3ac1ac54ecf843 | 348 | 7.5 | propext |
| s5i5 | 24d450734f273168 | 385 | 8.4 | propext |
| sb26 | 926b62d9c2e7eead | 359 | 6.0 | propext |
| sc25 | b2689cfc605f9a41 | 609 | 10.4 | propext |
| sk48 | 54a19b11042fe8c8 | 72 | 3.6 | propext |
| sp80 | f60c20bafd851f89 | 150 | 6.3 | propext |
| su15 | fc290b015a01deca | 81 | 5.6 | propext |
| tn36 | d829e8a253abae6f | 18 | 1.1 | propext |
| tr87 | efc52d00cdb8da9c | 38 | 2.0 | propext |
| tu93 | fd131f2c2774deec | 28 | 1.6 | propext |
| vc33 | ec86c7e7a0defa0b | 30 | 1.7 | propext |
| wa30 | e367f8f2fab17834 | 478 | 23.1 | propext |

**A.5 The direction of the guarantee.** On the covered domains, let $h : \mathcal{S} \to \mathcal{W}$ map concrete states, augmented with observation history where required, to modeled states, and let $V$ be the set of visited state–action pairs. Agreement with the record gives: for $(s, a) \in V$, step$(h(s), a)$ agrees with $h(T(s, a))$, and nothing outside $V$. If the model over-approximates the environment, with matching initialization, every relevant real transition represented, real goals mapped to model goals, and cost/budget semantics that do not exclude a cheaper real execution, then (Aineto & Scala, 2024; Clarke et al., 1994)

$$\text{Unsolvable}_{\widehat{\mathcal{M}}}\big(h(s_0)\big) \implies \forall\, \bar{a}.\ T^{\bar{a}}(s_0) \notin G.$$

Observation checks establish agreement only on the transitions and components they cover; they do not establish the over-approximation premise required for impossibility transfer. AWM therefore

Table 2: The formal side of each game's model. Level count and certified action count come from the ARC Prize scorecard; the remaining columns are computed from the repository. "Rules (Lean)" counts the rules translated into Lean. "Steps explained" is counted over the whole record, including exploration, so it can exceed the certified action count; for most games it counts the steps on which the Lean model's replay agrees with the record, and for a few it counts the steps with no misjudgment at the rule level. "Levels replayed" counts the levels whose recorded winning run Lean replays from the first screen up to the winning frame, whose step is checked by the model's goal test (formal replay coverage). †Mechanisms taken from the game's public source code were given to the agent during play: `su15`'s seal record states this, and for `sp80` it is recorded in the run's session log. The concepts these mechanisms brought in do not count as discovered from play.

| Game | Lv. | Cert. steps | Obj. types | Rules (Lean) | Rules total | Steps expl. | Levels replayed | Lean thms | Concepts (withdrawn) |
|---|---|---|---|---|---|---|---|---|---|
| `ar25` | 8 | 253 | 6 | 5 | 8 | 253/253 | 8/8 | 43 | 17 (3) |
| `bp35` | 9 | 374 | 3 | 10 | 11 | 348/374 | 9/9 | 41 | 26 (4) |
| `cd82` | 6 | 70 | 7 | 10 | 11 | 657/661 | 6/6 | 81 | 44 (13) |
| `cn04` | 6 | 261 | 4 | 8 | 9 | 742/745 | 6/6 | 51 | 22 (1) |
| `dc22` | 6 | 1,037 | 8 | 14 | 17 | 2664/2664 | 6/6 | 77 | 48 (6) |
| `ft09` | 6 | 80 | 6 | 4 | 7 | 168/168 | 6/6 | 50 | 15 (6) |
| `g50t` | 7 | 288 | 10 | 11 | 13 | 280/280 | 7/7 | 64 | 20 (1) |
| `ka59` | 7 | 421 | 9 | 10 | 11 | 3959/3959 | 7/7 | 67 | 22 (8) |
| `lf52` | 10 | 905 | 12 | 10 | 11 | 896/905 | 10/10 | 64 | 47 (6) |
| `lp85` | 8 | 79 | 6 | 4 | 6 | 199/199 | 8/8 | 47 | 23 (4) |
| `ls20` | 7 | 368 | 12 | 18 | 21 | 1460/1465 | 7/7 | 42 | 22 (3) |
| `m0r0` | 6 | 215 | 8 | 8 | 9 | 485/486 | 6/6 | 67 | 26 (5) |
| `r11l` | 6 | 81 | 6 | 8 | 9 | 183/184 | 6/6 | 65 | 26 (4) |
| `re86` | 8 | 547 | 7 | 8 | 12 | 2153/2153 | 8/8 | 42 | 40 (4) |
| `s5i5` | 8 | 461 | 5 | 17 | 19 | 2617/2618 | 8/8 | 70 | 49 (9) |
| `sb26` | 8 | 124 | 6 | 12 | 16 | 424/424 | 8/8 | 53 | 20 (1) |
| `sc25` | 6 | 146 | 5 | 11 | 18 | 382/401 | 6/6 | 31 | 27 (1) |
| `sk48` | 8 | 263 | 6 | 4 | 7 | 1364/1407 | 8/8 | 66 | 39 (8) |
| `sp80`† | 6 | 138 | 3 | 6 | 10 | 353/371 | 6/6 | 25 | 58 (23) |
| `su15`† | 9 | 90 | 5 | 12 | 16 | 783/904 | 9/9 | 64 | 37 (15) |
| `tn36` | 7 | 125 | 5 | 4 | 7 | 292/292 | 7/7 | 28 | 33 (9) |
| `tr87` | 6 | 119 | 6 | 4 | 7 | 429/429 | 6/6 | 50 | 13 (3) |
| `tu93` | 9 | 187 | 8 | 8 | 10 | 560/566 | 9/9 | 76 | 17 (0) |
| `vc33` | 7 | 167 | 9 | 7 | 10 | 535/538 | 7/7 | 75 | 14 (0) |
| `wa30` | 9 | 649 | 8 | 7 | 8 | 1515/1515 | 9/9 | 35 | 37 (6) |
| Total | 183 | 7448 | 170 | 220 | 283 | 23701/23961 | 183/183 | 1374 | 742 (143) |

states its impossibility results as Unsolvable of the inferred model. The failure in §4 is a violation of the over-approximation premise in the unsafe direction: an abstraction that forbids a route the engine allows. §4.3 reports where this premise fails and how the agents repaired the rule at fault.

**A.6 Structure & Simplify across games.** The runs reported in this paper apply generalization and deletion; the released code implements all four transformations. Table 3 lists, for each game, the size of the concept book, of the derived layer and of the final rule layer, the shared rule that absorbed the most concepts, and the comparison on corrupted steps where one was made. In 24 of the 25 games the per-game report states at least one many-to-one merge; `cd82` gives only the reason for a rebuild. In the per-game reports, which describe more merges than the largest one listed in the table, translation rules recur in 16 games, win conditions written as one equivalence in 10, counters and budgets in 7, switches in 6, and permutations in 3 (`lp85`, `dc22`, `ls20`). Where the book and the final rules were compared on the same corrupted steps (14 games), the rules reject more in 12, of which 10 reject a superset; one comparison is saturated (`tr87`); and in `dc22` the rules reject 2 fewer of 400, 26 steps being rejected only by the book. Condition (A.2) flagged 186 rules across the 25 games; none was removed on that ground alone, and each stays flagged in its game's record. Most reports give the reason for keeping, usually that the corrupted set is too coarse to single the rule out, and those of `lp85`, `dc22`, `g50t` and `re86` show each flagged rule rejected on its own by a source-code mutant.

Table 3: Structure & Simplify in each game, from the per-game reports (bench `fa928185`). *Book*: concepts at the start of Structure & Simplify; *Derived*: entries in the model's derived layer, which computes composite quantities from the object layer (`derive/`); *Rules*: rules in the final layer (these per-game counts sum to 287, against the 283 of §3, which are read at the paper's build). *Merged*: concepts the largest generalization absorbed. *Rejected*: corrupted steps rejected by the book and by the final rules, with the size of the set in brackets; the set differs across games, and in `cn04`, `dc22`, `sp80` and `tn36` it is the full corrupted corpus (400 to 1,804 steps), not the at most 120 used for deletion, and in nine games only the rules side counts steps the object parser refuses. *Change*: book against final rules; "int." marks games that only compare intermediate rule versions. [a]Counts intermediate rules. [b]One concept per direction split into three rules. [c]Absorbed into a group of seven rules. [d]Not a superset: the book alone rejects one reset step. [e]The report does not state it; concepts minus withdrawn concepts in Table 2, which gives the Book value of every other game.

| Game | Book | Rules | Derived | Largest merge | Kind | Merged | Rejected | Change |
|---|---|---|---|---|---|---|---|---|
| `ar25` | 14 | 8 | 6 | `push_moves_selected` | translation | 4 | — | int., equal |
| `bp35` | 22[e] | 11 | 2 | `sideways` | translation | 3 | — | int., equal |
| `cd82` | 31 | 11 | 6 | (rebuilt) | — | — | 115/117 (120) | up |
| `cn04` | 21 | 9 | 5 | `move` | translation | 9 | 377/399 (400) | up |
| `dc22` | 42 | 17 | 7 | `move` | translation | 6 | 329/327 (400) | down |
| `ft09` | 9 | 7 | 4 | `level_up_iff_all_done` | win condition | 2 | 116/118 (120) | up |
| `g50t` | 19 | 12 | 4 | `walker_patrols` | patrol | 4 | —/95 (120) | — |
| `ka59` | 14 | 11 | 7 | `push_shoves` | translation | 4 | 92/119 (120) | up |
| `lf52` | 41 | 11 | 3 | `keys_push_carts` | translation | 9 | 78/117 (120) | up |
| `lp85` | 19 | 6 | 5 | `press_permutes` | permutation | 11 | 117/119 (120) | up |
| `ls20` | 19 | 21 | 6 | `walk/slide/blocked` | translation | 4[b] | 105/118 (120) | up |
| `m0r0` | 21 | 11 | 5 | `pieces_move` | translation | 6 | — | int., up |
| `r11l` | 22[e] | 9 | 8 | `absorb` | region fill | 3[a] | — | int., equal |
| `re86` | 36 | 12 | 3 | directions a1–a4 | translation | 4 | —/119 (120) | — |
| `s5i5` | 40 | 19 | 4 | void-step group | other | 9[c] | —/117 (120) | — |
| `sb26` | 19 | 16 | 3 | `win/submit` | win condition | 2 | 116/119 (120) | up |
| `sc25` | 26 | 18 | 10 | `resize` | other | 4 | 118/119 (120) | up[d] |
| `sk48` | 31 | 7 | 2 | `rope_slide` | translation | 2[a] | — | int., equal |
| `sp80` | 35 | 13 | 7 | `arrow_shifts_selected` | translation | 6 | 1580/1778 (1804) | up |
| `su15` | 22 | 16 | 5 | `click_lands_fresh` | other | 2[a] | — | int., equal |
| `tn36` | 24 | 7 | 4 | `piece_follows_program` | program | 7 | 1390/1424 (1434) | up |
| `tr87` | 10 | 7 | 3 | `frame_rest` | static | 3 | 120/120 (120) | equal |
| `tu93` | 17 | 10 | 4 | `player_moves` | translation | 2[a] | — | int., equal |
| `vc33` | 14 | 8 | 5 | `press_button` | other | 3[a] | — | int., equal |
| `wa30` | 31 | 10 | 9 | `moles_act` | pathfinding | 10 | 85/120 (120) | up |

*Proof size before and after Structure & Simplify (`lp85`).* To test whether the merged rule makes proving easier, we give Claude Opus 5.5 34 `lp85` proof tasks (eight original levels and 26 modified or generated instances), each to be proved winnable (with a winning click sequence) or unwinnable (with a Lean proof), once with model A, the eleven concepts written during play, and once with model B, the single `press_permutes` rule; three repetitions per task and model, 204 runs in all. The instances are the eight original levels; the eight levels with one goal colour removed, which cannot be won; a swap that makes level 6 unwinnable and a second swap that makes it winnable again; six generated large layouts (76–129 cells, against at most 75 in the game) with an unwinnable variant of each; and four generated layouts whose only obstruction is the spacing of an isolated ring. Every verdict is checked against exhaustive search on the engine. Model B is proved in 102 of 102 runs and model A in 99; no run gives a wrong answer, and A's three failures are two timeouts and one run that stopped without an answer. Table 4 gives the proof sizes; on winnable instances both models are proved by a click sequence and `decide`.

**A.7 Unexplained steps.** Of the 23,961 recorded steps, 260 are not explained by the rules (§3). Table 5 sorts them by cause. 3 are wrong predictions, 2 of them in `su15`, none on a winning route; on the other 257 the model gives no verdict. The largest groups are undo, which needs history the screen does not hold (100), and steps that reset or change the level (78). State the screen does not show accounts for 70: a facing direction in `sk48`, a piece that drifts out of the visible window in `bp35`, overlapping pieces in `sp80`, fog in `ls20`. Several pieces acting on each other in one step account for one, in `su15`.

Table 4: Proving 34 `lp85` tasks with the eleven concepts (A) and the merged rule (B): runs proved, and medians of lemmas, proof lines and kernel time over the runs of each group.

| Instances | Runs | Proved A / B | Lemmas A / B | Lines A / B | Kernel (s) A / B |
|---|---|---|---|---|---|
| original levels (winnable) | 24 | 24 / 24 | 0 / 0 | 4.5 / 4 | 1.00 / 0.83 |
| goal colour removed (unwinnable) | 24 | 24 / 24 | 6 / 3 | 75.5 / 28 | 0.90 / 0.80 |
| level 6 swaps (one each) | 6 | 6 / 6 | 2 / 1.5 | 36.5 / 24 | 1.69 / 1.22 |
| large layouts (winnable) | 18 | 16 / 18 | 0 / 0 | 5.5 / 4 | 1.95 / 1.00 |
| large layouts, altered (unwinnable) | 18 | 17 / 18 | 12 / 4 | 117 / 49.5 | 2.82 / 2.41 |
| ring spacing (unwinnable) | 12 | 12 / 12 | 12 / 6.5 | 111 / 62.5 | 4.27 / 3.02 |
| all | 102 | 99 / 102 | | | |

Table 5: The 260 recorded steps the rules do not explain, by cause (bench `fa928185`). Counting follows each game's report: Lean step agreement in most games, the rule layer in `s5i5` and `m0r0`.

| Cause | Steps | Games |
|---|---|---|
| Undo: needs history | 100 | `su15` 92, `bp35` 8 |
| Reset, new game or level change | 78 | ten games |
| State not visible on the screen | 70 | `sk48` 35, `bp35` 18, `sp80` 12, `ls20` 5 |
| Several pieces interact in one step | 1 | `su15` |
| Trigger not identified, branch left out | 5 | `su15` 1, `sp80` 4 |
| Attributed but unfixed error | 1 | `s5i5` |
| One side is a win screen the parser cannot read | 5 | `sc25` 3, `m0r0` 1, `sk48` 1 |

## B The budget-sealed instance: construction, protocol and known gaps

This appendix records the construction, protocols and audits behind §4. The construction history and earlier batches are omitted; the evaluation of §4 uses the runs identified in §B.7 and §B.6.

**B.1 Construction.** The order was fixed in advance: locate the field that stores the budget and confirm that editing only it changes only the budget; build the abstract models and cross-check them against the engine edge by edge; search for $B^\star$, the cost of the cheapest win; seal the level at $B^\star - 1$; certify $B^\star$; and only then start the runs. The cross-check precedes the search because the abstract model evaluates an edge in microseconds where the engine takes milliseconds, and the certification precedes the runs because one wrong clause in the search semantics would make the instance fake without announcing itself. Section 4 relies only on the engine-level exhaustive search for $B^\star$.

**B.2 Configurations and agents.**

- **baseline (no AWM)**: task and interface with the coding agent's ordinary tools; no required AWM procedure or supplied formal skeleton. In §4 these are Claude Opus 5.5 and GPT-6 Astra (§B.4, §B.5). This is the control: whether it decides unsolvability, and what it offers as warrant, is what the experiment measures.
- **framework**: the AWM procedure, a generic Lean skeleton, and tools for recording predictions and checking rules, but no ready-made model.
- **solvable side**: the same board with level 5's cap restored, as the non-vacuity gate for the configurations themselves.

Arena directories carry opaque names with no marker of configuration or side, the arenas live outside the project tree, no ground-truth file enters an arena, and cross-run memory is cleared before an independent run starts; state and files persist between attempts of that run, with the entering file count required to be 0. Runs interrupted by machine failures (two restarts during the batch) were rerun from the start; their partial arenas were archived and are not counted. All runs reported in §4 use the public rule code; the corrected engine is used only in a level-4 control of three GPT-6 Astra runs without AWM, which is not reported here. Blinding lists were extended to this level's constants and files before the runs, and both arenas were scanned afterwards with no ground-truth file found.

*Agents.* All agents observe screens and submit actions through the same interface in isolated workspaces, and each attempt ends with an interview that records a verdict and its argument. The **framework** agent receives the AWM procedure, a generic Lean skeleton and tools for recording predictions and checking rules, but no game rules. **Baseline** agents (no AWM) receive only the task and the interface, with the ordinary tools of a coding agent. Codex runs used in §4 were checked turn by turn to have a writable workspace; earlier runs without one are not used.

**B.3 Engine ground truth.** Table 6 summarizes the two sealed levels.

Table 6: The two sealed levels. Costs are in counter units; "gross" adds back the fuel refund of 10. Reachable states are those of the engine at the sealed cap, none of which is won.

| Level | Public cap | Sealed cap | $B^\star$ (gross) | Reachable states | Win at $B^\star$ |
|---|---|---|---|---|---|
| 4 | 35 | 18 | 19 (29) | 656,715 | 27 actions |
| 5 | 65 | 35 | 36 (46) | 1,346,362–1,354,849[a] | 42 actions |

[a] Per opening; the opening facing carries over from the previous level and cannot be read from the screen, so level 5 was searched from all four (below).

The search runs on the sealed engine itself. Its actions are the four moves and the nine panel coordinates that the engine exposes, clicked one at a time; recipes are not abstracted. The state key is every sprite except the counter bar, the panel display and the spell icons (name, position, scale, rotation, pixels, visibility), together with the panel matrix, the facing, the animation flags, the teleport target index and the game state. The counter is not in the key: each key keeps the least count at which it is reached, in a label-correcting search where a refund may reopen a state. This is sound because the counter enters the engine only through its unit increment, the refund $\max(0, c - 10)$, the loss test

*count* $>$ *cap*, and the display. A fast state copy agreed with a deep copy on 6,000 steps, a rerun after an externally killed first run reproduced the state and edge numbers of every count from 0 to 23, and clicks on all 4,096 screen coordinates in 22 sampled positions had no further effect.

The opening facing is a game-level attribute: a level reset does not reset it, the opening frame does not show it, and all four facings occur in the service. The search was therefore run from each opening:

| opening | states, cap 35 | engine edges, cap 35 | wins | states, cap 36 | wins |
|---|---|---|---|---|---|
| the one `set_level` produces | 1,346,362 | 17,515,680 | 0 | 1,371,613 | 1 |
| each of the other three | 1,354,849 | 17,626,011 | 0 | 1,380,256 | 1 |

The 42-action win is

```
D D R c1 c4 U c7 R R R R D R R R U U U L c1 c3 c5 c7
  U U U U R c1 c3 c5 c7 c1 c4 c7 c0 c1 c4 U U U U;
```

the searches from the other openings return 42-action wins that differ from it only in where one upward move falls among the first clicks. It is 25 moves and 17 panel clicks that complete five recipes. Each completing click is charged one extra unit, and the final move into the goal is not charged at all and is allowed when the count equals the cap, so the gross charge is $46$ and, after the refund of $10$, the count peaks at $36$. The original engine with its cap set to 36 accepts the same sequence.

*Level 5 ground truth.* The ground truth is an exhaustive search on the real engine, not a model of ours. The level's opening facing is not fixed: it is carried over from the previous level, is not reset with the level and cannot be read from the opening frame, and all four facings occur in the service. We therefore searched from all four openings. At cap 35 every reachable state is explored and none wins: 1,346,362 states linked by 17,515,680 engine edges from one opening, and 1,354,849 states linked by 17,626,011 edges from each of the other three. At cap 36 every opening admits a win. A 42-action winning sequence, replayed in a fresh engine process, clears the level at cap 36 on its 42nd action and ends the game at cap 35 on its 41st. The optimum is therefore $B^\star = 36$ from every opening, and the sealed cap is one unit below it. It does not depend on a refund rule of the public engine that matters on level 4 (§4.2): our model of the level, cross-checked against the engine, gives 36 under either form of that rule.

*Engine truth used for checking.* The experiments of §B.10 and §B.11 and Appendix C are checked against a fresh exhaustive expansion of both sealed engines. The search represents the engine's dynamic configuration and its counter separately: the configuration key omits the counter, while budget information retains its effects on transitions and terminal checks. Leftover animation parameters and a sprite-sorting cache are normalized, with tests showing that swapping them changes no outcome. The search covers all four opening facings, since the facing carries over from the previous level, and all 13 actions (four moves and nine panel cells) from every reachable configuration, 905,587 on level 4 and 1,423,609 on level 5. It covers every write to the counter in the source: six writes (two resets, the move or spell completion and the panel click, each $+1$, and the two fuel refunds) and two checks against the cap, each with tests and edge counts; the refund's clamp at zero is unreachable within the sealed caps. Clicks outside the nine panel cells were scanned over the whole $64 \times 64$ screen on 30,001 sampled frame classes per level, 3.35% of 894,948 on level 4 and 2.16% of 1,391,503 on level 5: in these scans, apart from the panel, only the spell icon changes the state, and it changes none of the 13 actions' outcomes. Exhaustiveness of the expanded graph is relative to the registered 13-action abstraction; coverage of other screen clicks relies on source analysis and the stated sampled scans rather than a full scan at every reachable frame class. At the sealed caps no opening has a winning edge; $B^\star$ is 19 on level 4 and 36 on level 5 from all four openings, with optimal routes of 27 and 42 actions, which win on the unmodified engine at cap $B^\star$ and die at cap $B^\star - 1$. A state-by-state engine search to depth 9 and 2,000 sampled deep states per level agree with the derived graph on every edge.

**B.4 Level 4 runs.** The level-4 runs of Table 2 were made under the original wording: three eight-attempt runs for each of three agents (Table 7). No Claude Opus 5.5 run without AWM asserted impossibility, in any attempt or in the final interview. A longer, thirteen-attempt Claude run asserted it at attempt 9, returned to `winnable` at attempt 11 and asserted it again at attempts 12 and 13, on a gross cost of 32 against the true 29. Two of three GPT-6 Astra runs without AWM, whose session logs

Table 7: The level-4 seal: the runs of Table 2, eight attempts per run, original wording. *Asserted*: runs with an unqualified `impossible` (baseline) or an impossibility verdict (framework) in some attempt, and the attempts at which they first did. *Stated*: the cost and budget a baseline run gave when it asserted; the truth is 29 against 28.

| Engine | Agent | Runs | Asserted | Stated |
|---|---|---|---|---|
| public | Claude Opus 5.5 | 3 | 0 | — |
| | GPT-6 Astra | 3 | 2 (8, 8)[b] | 32 against 28, both; one adds 29 under a loosened rule set |
| | AWM | 3 | 3 (1, 2, 3)[a] | Lean, clean axioms[a] |

[a] One run returned to `winnable` at attempts 4–5. Rebuilt with other caps, the proof of the run whose Lean bound is 20 holds up to cap 19 and fails at 20; the Lean file of the run whose bound is 19 proves that no list wins up to cap 18 and gives a win at 19, so its range is 18 (its theorems cover every cap up to 18, and the file recompiles); the third run's (`ff069f`) universal theorem rests on a check that was not compiled to completion, so its range was not measured; in Table 2 this run is replaced by `l4p-case-2a3cc5`, whose final Lean development was rebuilt and checked independently (§B.6). [b] One of these runs returned to `undecided` in the final interview.

show that model and a writable workspace throughout, asserted it at attempt 8, each on a cheapest cost of 32 (one in half units, 64 against 56). One returned to `undecided` in the final interview; the other kept the verdict and added that a search under a deliberately loosened rule set "still requires 29", the true optimum. The three framework runs first reported impossibility at attempts 1, 2 and 3, one of them returning to `winnable` at attempts 4 and 5; two of them submitted Lean that compiles with clean axioms, and the third (`ff069f`) a universal theorem whose check was not compiled to completion; in the main cohort of §4 it is replaced by `l4p-case-2a3cc5`, whose final Lean development was rebuilt and checked independently (Table 7).

**B.5 Level 5 runs.** The level-5 baseline runs of Table 2 were made on 2026-10-18 with new case identifiers, each building a fresh arena, under the original wording: Claude Opus 5.5 and GPT-6 Astra without AWM, three runs each, eight attempts each. Every Codex attempt was checked in its session log to have run `gpt-6-astra` with a writable workspace. One Codex run stopped at attempt 7 when the provider reported the model at capacity; its first six verdicts were `undecided`, it is not counted, and it was replaced by a spare run. All three Claude Opus 5.5 runs without AWM asserted impossibility, at attempts 4–6; when a baseline run asserts it, the argument is a hand count that a reader cannot check.

The three framework runs of Table 2 on level 5 were made on 2026-10-20 with new case identifiers, fresh arenas and asks identical to those of the baseline runs. The framework's method contains a section on closing the proof: the bound is to be proved in Lean over the model itself, the theorem quantifying over every action list and the tables it rests on checked by the kernel; the only hypotheses allowed restrict which actions are legal; and a second theorem shows that loosening one limit of the model by a stated amount admits a win. It also advises changing the model's representation when its table is too large to check, and treating a failure to close as a reason to revise the model. The section and the criteria for calling a proof closed were preregistered before launch. All three runs closed their bound; they are the framework rows of Table 2.

The three closed theorems were rebuilt from source in a fresh directory and use only `propext`, `Quot.sound` and, in one run, `Classical.choice`. In `l5d-case-4f4628` the theorem also takes as a hypothesis the model's own budget law, an inequality over the numbers of actions, completed spells and refills in the run; we count it as a rule of the model, not as a computed fact. Replayed from each run's server log, every recorded level-5 transition, 280 to 319 per run, is reproduced by the run's final Lean model and satisfies its certificate. Checked against the 17,194,487 engine transitions reachable at cap 35, no model contains the engine: among other gaps, each omits the cells of an opened barrier, which a block may enter on the engine. On the transitions between states it covers, the engine respects the certificate of `4f4628` on every edge, that of `l5d-case-284967` on all but 8 grow edges, each one unit cheaper on the engine, and that of `l5d-case-977714` on all but 111 edges, the engine's shot from below the first barrier, which its model prices 12 to 14 units higher. Evaluating each model's Lean definitions on the engine's cheapest route locates the excess. In `284967` the route runs out of budget exactly on the winning move, and a variant that does not charge that move wins one unit above the cap, as the engine does. In `977714` the route never wins until the shot distance is widened, after which it wins at 37, and the last unit is again the winning

move. During one run's second attempt the memory guard stopped three of its compilations; its final theorem was compiled afterwards.

**B.6 The six AWM runs of Table 2.** The six AWM results in Table 2 are listed in Table 8. Correctness is assessed on the bound reported before feedback. As in §4.2, a bound is correct if it equals the engine optimum (19 on L4, 36 on L5) or differs from it only by charging the winning move, which the engine waives and which cannot be observed on a sealed level; this is a cost-accounting convention, not a numerical tolerance. Four of the six are correct, among them the L5 run that reported 37; the two runs whose bounds were 20 and 47 are the follow-ups of §B.7, and after them all six bounds are correct (19, and 37 with the winning move charged).

Table 8: The six AWM runs of Table 2: the attempt of the first impossibility claim (mean 2.0), the bound reported before feedback, and the model proof.

| Level | Case suffix | First claim | Bound | Correct bound | Model proof |
|---|---|---|---|---|---|
| 4 | `9ec8d4` | 2 | 20 | no | yes |
| 4 | `69fe87` | 1 | 19 | yes | yes |
| 4 | `2a3cc5` | 2 | 19 | yes | yes |
| 5 | `284967` | 3 | 37 | yes | yes |
| 5 | `4f4628` | 2 | 36 | yes | yes |
| 5 | `977714` | 2 | 47 | no | yes |

Model proofs are counted for their stated definitions and explicit premises, independently of numerical correctness.

*Scope of the proof count.* The 6/6 count concerns final model-relative impossibility developments, not six formally transferred engine certificates. For example, `9ec8d4`'s original bound of 20 belongs to `M15fwL4.lean`; later files in the same arena prove 19 and must not be used to describe that original result. The theorem in `4f4628` retains an explicit arithmetic budget-law premise, and other developments state their legal-action or rule restrictions. A closed proof discharges the theorem under these premises; it does not verify the premises against every engine behavior. Transferring impossibility within a budget requires every real winning execution within that budget to have a winning modeled counterpart within that budget. Agreement on recorded transitions, or even a correct numerical threshold, does not by itself establish this coverage.

*Timing.* First-claim attempts are $2, 1, 2, 3, 2, 2$ (Table 8), averaging 2.0. This records judgments, including later withdrawals, and is not a proof-completion metric. Per-attempt compiler and axiom logs permit some proof events to be recovered, but theorem names alone do not fix the model version. In `9ec8d4`, a conditional impossibility theorem already passes at attempt 1 while the verdict remains undecided; the first impossibility claim is at attempt 2. In `284967`, attempt 2 proves only a no-grow restriction; the general budget lower-bound theorem is checked at attempt 3. Neither an arbitrary successful Lean compilation nor an unchanged theorem name is sufficient to date the final-bound proof.

**B.7 Repair after the eight attempts.** The repairs of §4.3 follow the eight attempts of each run and use one generic signal: the engine has a cheaper winning route than the run's analysis allows, because at least one assumed rule forbids, or charges more for, a move the engine allows; no rule, cost or route is named. The two AWM runs and the four baseline runs with wrong bounds received it in the same round; the text of the message differs slightly between the arms, as follows. Table 9 lists the feedback each run received. No rule was named to any run in this round.

*AWM (`m15fw-case-9ec8d4`, level 4; `l5d-case-977714`, level 5).* Each follow-up starts from its run's saved post-evaluation session and workspace. The generic signal is followed by a self-audit instruction: relax rules consistently with the interaction record, identify relaxations that lower the winning cost, test their first disagreement with the original model, then revise, replay and prove again. Both selected repairs use four further attempts on the target level.

On L4 the four reported bounds are 21, 19, 19 and 19: the correct bound first appears at attempt 2. Its final theorem excludes all modeled wins within 18 under explicit premises, with a winning witness at 19; the revised proofs were independently rebuilt. On L5 the follow-up's service log records four attempts on level 5 (104 actions, one game over), although it is stored as one aggregate CLI

Table 9: Feedback in the repair round of §4.3: one round per run, the same discrepancy signal in both arms, no rule, cost or route named.

| Run | Level | Feedback | Bounds reported after it | Proof |
|---|---|---|---|---|
| AWM `9ec8d4` | 4 | signal + self-audit instruction (relax, test, revise, prove again) | 21, 19, 19, 19 | yes |
| AWM `977714` | 5 | signal + self-audit instruction | 37 | yes |
| Baseline `44ec53`, `4d3b31` | 4 | signal + the same logic in plain language | 19 (both) | no |
| Baseline `339b0a`, `202da3` | 5 | signal + the same logic in plain language | 42 (of ∼40), 68 (of 64) | no |

continuation. It reports 37, marking the winning-move charge untestable, and its independently rebuilt theorem excludes all modeled wins within 35. Thus both repairs finish within four additional attempts; CLI turns or interview messages are not the attempt-counting unit.

Both follow-ups resume the original runs' sessions.

*Baseline (no AWM) (`bast2-case-44ec53`, `bast2-case-4d3b31`, level 4; `l5r-case-339b0a`, `l5r-case-202da3`, level 5).* Each run resumes its own session after its final interview, with the service restarted from level 1. The signal adds that it says nothing about whether the level can be won within its budget, and a plain-language paragraph with the same main logic as the self-audit: a rule stricter than the game makes the least cost too high, so the rules the least cost depends on should be tested in the game, especially in situations never played. At most four further attempts are allowed; the driver resets the level at the start of each and an attempt ends at a game over; after each the run reports its least budget and verdict. No run reset the level itself. Both level-4 runs report 19 from their first further attempt on (one in energy units of two per move, 38 against a measured 36), and both end `undecided`; neither bound is proved. On level 5 the runs end at 42 against a measured effective budget of about 40 (`undecided`) and at 68 against 64 in rows of the budget bar (`impossible`); neither states the engine's optimum in its own units. Read with a conversion by hand, two actions beyond the budget of which the second enters the goal, both correspond to 37, the engine's 36 plus the winning move; we do not count them, because the conversion is ours and the runs' own budgets are approximate. Replayed on the engine, every baseline run played at least one route of the engine's least cost during the round and stopped one uncharged move short of the goal.

*The message in the two arms.* Both arms receive the same discrepancy signal. The AWM message points to the self-audit section of its method; the baseline message states the same main logic in plain language and adds that the signal does not settle whether the sealed level can be won. Baseline follow-ups and the L4 follow-up use driver-started attempts; the L5 follow-up counts the run's resets from service logs. Neither arm receives the correct bound or an engine-winning route.

**B.8 Checking what the bound depends on.** A Lean certificate establishes that a conclusion follows from the encoded transition system. It does not establish that the encoded system describes the environment. The framework agent's bound shows the difference. We translated its Lean model into Python, checked the translation against Lean's own evaluation on all 32,523 abstract transitions found by the exhaustive engine search of §4.1 with no disagreement, and compared the model with the engine. Two of its departures act on the bound. The wrong teleport gives the model a path the engine does not have, and the charge on the winning move adds one unit to every win. They offset. The modeled bound equals the engine's threshold, but by the engine's accounting the unrepaired model does not give 36: charging the winning move as the engine does lowers its bound to 35, along a path through the refuted teleport.

*Testing the region below the bound.* A model lower bound motivates checking its below-bound transition region. Agreement on that model-derived region alone does not ensure that every cheaper engine execution is represented: the engine may reach states or allow actions the model omits. Transfer also requires complete coverage under the abstraction, action-set, and initialization assumptions of the engine search. For the framework agent's model this region holds 20,853 (state, action) pairs. For each pair we started a fresh game, reached the state by real actions, and executed the action on the engine; 421 pairs could not be reached, and the region also contains 1,144 transitions that die at the cap. Completeness therefore rests on a final exhaustive check, not on the region. The procedure

used about 470,000 engine actions and recorded 3,184 disagreements (teleport 1,607, resizing 1,457, turning in place 48, multi-cell moves 26, other 46).

The preregistered repair only adds observed transitions. Under it the bound stays 36, but that 36 still includes the charge on the winning move. In an analysis added after the preregistered one, we replaced each refuted transition with the observed one and charged the winning move as the engine does, taking the winning move from an observed run at cap 36. This gives a bound of 36 again, and the repaired model passes an exhaustive check against all 32,523 abstract engine transitions within cap 35 with no violation; every transition on its cheapest modeled win also occurs in the engine. Two limits apply. Over the transitions within the cap, this certificate states the same fact as the exhaustive search; its value is that it is derived from the agent's own model and can be checked independently. And the repair writes observations back into the model; it does not learn general rules.

Lean is compared with an equally checked Python program in Appendix C.

**B.9 Interview prompts.** After each attempt the harness sends the interview below, in the same session as the play. All runs receive questions 1–3 and the verdict lines; AWM runs also receive the formalization questions, in the version of the harness they ran under, and baseline runs receive a verdict line without the model-relative answers. The level is then reset with the retry message. After the last attempt the final interview replaces questions 1–3 by its own four questions and keeps the same verdict lines. The texts are reproduced verbatim.

*Questions 1–3, every run.*

```
That attempt failed. The game will reset this level, and you may get another attempt.

Answer the following. Keep it to what it needs; do not pad.

1. What happened on the level you were on when the attempt ended, and what do you think made
   it fail? Explain in plain prose, as if to someone who did not watch you play.

2. Do you still see a way to beat this level?
   - If you do: say what it is, and what you would still need to check to be sure of it.
   - If you no longer do: say why not. State precisely what you are claiming, give the
     argument for it, and make clear what the argument rests on - which parts you established
     and which are assumptions you did not check. Someone else should be able to follow it and
     judge whether it holds. The form of the argument is yours to choose.
   - If you cannot tell either way, say so, and say what would settle it.

3. What will you do on the next attempt?
```

*Formalization questions, AWM runs of the later harness.*

```
4. Before you answer this one, bring your formal system up to date. It is the thing you are
   building across attempts; the proof is downstream of it. Work in this order, in your
   working directory:

   a. Objects. What things are on the board, and what slots does each thing carry? For every
      object you keep, you should be able to name a pair of positions that stop being
      distinguishable if you drop it.
   b. Rules. Write each rule as a function: given a position and an action, what does it
      change? Each rule needs a domain you can actually execute.
   c. Simplify. Drop slots no rule reads. Merge rules that always decide alike. After this,
      your model's verdict on every frame you have observed must be unchanged.
   d. Formalize. Turn the objects and rules into Lean *definitions* - not theorems. Each rule
      ends up in exactly one of three states: a definition, a consequence of definitions, or
      explicitly marked "not in the model".

   If you already have a formal system from an earlier attempt, **amend it. Do not start a new
   one.** What you learned this attempt is a change to specific objects, rules or definitions;
   make those changes and leave the rest alone. A rewrite throws away the part of your work
   that was already right, and it destroys the record of what this attempt actually taught you.

5. Now prove your answer to question 2 **on that formal system**: that the level can be won,
   or that it cannot.

   - The proof must compile, and `#print axioms` on your main theorem must come back clean -
     no `sorryAx`, no `native_decide`. Report that output verbatim.
   - Say what your theorem actually quantifies over and what it assumes. If the proof needs a
     premise you did not establish, write it as an explicit hypothesis rather than burying it
     in a definition.
   - Name the rules from step 4b that your theorem leans on. If your Lean definitions carry
     something your rules do not - an extra assumption, a tighter domain, a step treated as
```

```
     atomic - say so here. A proof that is clean about the wrong system is worth nothing, and
     a model that is *stricter* than the game is the dangerous direction: "impossible in my
     model" does not transfer to the game.
   - Put the file in `lean/` and tell me its filename.
   - If you cannot complete the proof, say exactly where it breaks and what is missing. An
     honest "it does not go through, here is the gap" is worth more than a theorem that proves
     something weaker than it looks.

6. If this attempt taught you nothing that changes your formal system - no rule revised, no
   definition touched - do not invent a change. Say so instead, on its own line, in exactly
   this form, and then give the reason:

       FORMAL-UNCHANGED: <why this attempt added nothing to the model - what you tried, what
       you already knew, and what would have had to happen for the model to change>

   "No change, and here is why" is a real result. A change made only to look busy is not.
```

*Formalization question, AWM runs of the earlier harness.*

```
4. Whichever way your answer to question 2 goes, prove it in Lean now, using the formal
   apparatus you were given in this working directory: that the level can be won, or that it
   cannot.

   - The proof must compile, and `#print axioms` on your main theorem must come back clean -
     no `sorryAx`. Report that output verbatim.
   - Say what your theorem actually quantifies over and what it assumes. If the proof needs a
     premise you did not establish, write it as an explicit hypothesis rather than burying it
     in a definition.
   - Put the file in `lean/` inside your working directory and tell me its filename.
   - If you cannot complete the proof, say exactly where it breaks and what is missing. An
     honest "it does not go through, here is the gap" is worth more than a theorem that proves
     something weaker than it looks.
```

*Verdict lines, AWM runs.*

```
End your reply with exactly these two lines, on lines of their own, with nothing after them:

VERDICT: winnable | impossible | impossible_in_model_proved | impossible_in_model | undecided
CONFIDENCE: <an integer from 0 to 100>

The two "in_model" words are for when the level cannot be won under the rules you yourself
measured, and the step from "those rules" to "the real game" is an assumption you are stating
rather than something you established. Say which assumption, in your answer to question 2.
Use `impossible_in_model_proved` only if you are also handing over a Lean proof of it that
compiles and whose `#print axioms` is clean - name the file. Otherwise use `impossible_in_model`.
Your proof will be compiled and its axioms checked, so do not claim `_proved` on a file that
does not go through; `impossible_in_model` with an honest account of the gap is worth more.

Answer in writing only. Do not send any more game actions.
```

*Verdict lines, baseline runs.*

```
End your reply with exactly these two lines, on lines of their own, with nothing after them:

VERDICT: winnable | impossible | undecided
CONFIDENCE: <an integer from 0 to 100>

Answer in writing only. Do not send any more game actions.
```

*Retry message.*

```
You have another attempt on that level.

`POST /reset` restarts the level you were on; the levels you already beat stay beaten, and the
level's own counters start over. Play it again, as far as you can. When the attempt ends, stop
and report as before.
```

*Final interview, questions.*

```
That attempt failed.

Answer the following in writing. Give each answer the length it needs; do not pad.

1. What happened on the level you were on, and what do you think made it fail?
   Explain in plain prose, as if to someone who did not watch you play.
```

```
2. Do you still see a way to beat this level?
   - If you do: give the line of play, and say what you checked it against.
   - If you no longer do: say why not. State precisely what you are claiming, give the
     argument for it, and make clear what the argument rests on - which parts you established
     and which are assumptions you did not check. Someone else should be able to follow it and
     judge whether it holds. The form of the argument is yours to choose.
   - If you cannot tell either way, say so, and say what would settle it.

3. Which rule did you newly work out across these attempts, and from which observation?

4. Which rule are you still unsure about, and unsure in what way?
```

**B.10 Locating an injected error from the proof.** *Design.* We start from the reference model of each sealed level, cross-checked against the engine under the action and state abstractions of §B.3, and inject an error that changes a single rule. A sample *affects the conclusion* if the injection changes the model's least winning cost. The main sample is the 14 such injections on level 4 and 7 on level 5; injections that leave the conclusion unchanged and two error-free copies of each reference model are run as controls. Every sample is re-rendered in Lean with its own certificate, witness and theorem constants and stripped of comments, so that the samples cannot be told apart by their format. The agent (Claude Opus 5.5) receives the sample model with its proof and a probe service on the sealed engine, and is asked to find where the model and the engine disagree. The arms differ only in what else they receive: C1 a per-rule evidence table scored from the recorded transitions; C2 the same table and the proof's rule dependencies, the rules the theorem rests on as extracted from the Lean development, without the faulty one being named; R is told which rule is faulty. C0 takes states and actions uniformly at random without an agent, and EX searches the engine exhaustively without a budget. An observation counts as a *discovery* if it lies in the scope of the faulty rule and the engine disagrees with the sample model there; it is scored by the cumulative engine actions, resets included, up to the first discovery, independently of what the agent reports. Each sample runs with seeds 1–3 in every arm, paired by sample and seed. The action budget is 50 on level 4 and 5,000 on level 5, and the token cap 10.86M. Criteria and hypotheses were frozen before the runs; infrastructure failures were rerun once with the same seed and the original records are kept.

Table 10: Locating an injected error, main sample (14 injections on L4, 7 on L5, three seeds each). Found: discovery within the action budget. Median: Kaplan–Meier median of actions to the first discovery, censored at the budget; $> 50$ or $> 5000$ means the median is not reached within that budget. Named: runs whose conflict log or final model points to the faulty rule. EX has no budget; its row gives the discoveries over its full search.

| Arm | L4 found | L4 median | L5 found | L5 median | Found, both | Named | Tokens (median) |
|---|---|---|---|---|---|---|---|
| C0 random | 12/42 | $> 50$ | 0/21 | $> 5000$ | 12/63 | — | — |
| C1 per-rule evidence | 16/42 | $> 50$ | 1/21 | $> 5000$ | 17/63 | 35/63 | 2.35M |
| C2 proof dependencies | 29/42 | 27 | 3/21 | $> 5000$ | 32/63 | 50/63 | 3.29M |
| R faulty rule named | 30/42 | 26 | 5/21 | $> 5000$ | 35/63 | 61/63 | 1.98M |
| EX exhaustive | 14/14 | 136,852 | 1/7 | — | 15/21 | — | — |

*Results.* Table 10 gives the main readings. The paired difference in discovery between C2 and C1 over the 63 sample–seed pairs has mean 0.238, with a sample-clustered bootstrap 95% interval of $[0.079, 0.397]$ (0.310 on level 4, 0.095 on level 5). On level 4 the median number of actions to the first discovery is 27 for C2 and does not exist within the budget for C1. On the error-free controls C2 raises no false alarm (0.00 per run over 12 runs), against 0.33 for C1. Both preregistered hypotheses hold. The result rests mainly on level 4: on level 5 all agent arms rarely discover the error by probing within the budget. The main injected-error sample contains only cases in which the faulty rule lies in the supplied proof-dependency set. Results do not cover omitted enabling behavior whose absence leaves no corresponding rule in that set. On the controls whose injection leaves the conclusion unchanged, discovery on level 4 is C0 3/15, C1 3/15, C2 1/15 and R 8/15, and 0 in every arm on level 5.

*Final models.* Among the main-sample runs, C2 changes its model in 30 of 42 runs on level 4 and 19 of 21 on level 5, and its final theorem states the engine optimum in 30/42 and 19/21; for C1 the counts are 9/42 and 7/21, for R 29/42 and 16/21. All 110 final developments selected because their theorems state the optimum were rechecked with the Lean kernel: all compile, with no `sorry` and no `sorryAx` in the main theorem's axioms. This is a selected artifact check, not a 110/110 success

rate over all repair runs; absence of `sorryAx` alone is not a complete axiom audit. Checked against the engine truth, C2's final certificates on level 4 hold in 27 of 42 runs (C1 19/42, R 34/42).

*Faulty models produced by AWM.* Two faulty level-4 models came from AWM's own runs: one whose stated bound is 20 (faulty rules for growing and moving), and one whose stated bound is 22 (faulty rules labelled `SHRINK` and `SIX`). With the same protocol, budget 50 and seeds 1–3:

| Sample | C0 | C1 | C2 | R | EX (no budget) |
|---|---|---|---|---|---|
| bound 20 | 0/3 | 2/3 (31, 20) | 3/3 (10, 11, 15) | 1/3 (14) | found, 2,988 |
| bound 22 | 1/3 (15) | 2/3 (17, 13) | 1/3 (9) | 1/3 (9) | found, 4,256 |

Numbers in brackets are the actions to the first discovery. One C1 run on the first sample crashed after one engine action and is counted as not found.

**B.11 AWM against the same agent without it.** On the two sealed levels, three arms receive the same task statement, the same base model, eight attempts and a token cap of 150M per run, five runs per arm and level. A is the agent without AWM, with coding tools. B follows the AWM procedure up to an executable model and a Python solver, without Lean. C is the full AWM procedure with Lean. B and C also receive the prior that the rules stay fixed, the configuration of the original level-4 batch; A does not. The bound of a verdict is read from the main Lean theorem for C, from the run's own solver for B, and from the text for A, and converted to the engine's accounting along the run's own route; a bound is *equal to* $B^\star$ if it equals 19 or 36, and *holds on the engine* if it does not exceed $B^\star$.

| Arm | Verdict reached | Bound $= B^\star$ | Holds on the engine | Tokens per run (median, L4 / L5) |
|---|---|---|---|---|
| A without AWM | 4/10 | 0/2 | 0/2 | 10.9M / 10.7M |
| B AWM without Lean | 9/10 | 3/9 | 5/9 | 57.5M / 40.8M |
| C full AWM | 9/10 | 2/8 | 4/8 | 45.9M / 42.0M |

Verdict counts include claims made during a run. Bound denominators include only non-withdrawn verdicts whose bounds can be converted; final kernel-checked proof completion is a separate outcome. A reaches a verdict only on level 4 (4/5) and never on level 5; B and C reach one in 5/5 runs on level 4 and 4/5 on level 5. In C, three of the five level-4 runs and all four level-5 runs with a verdict end with a kernel-checked main theorem over all action sequences with clean axioms (seven final proofs in total, distinct from the nine verdicts); in B, every solver reruns and prints a well-formed result. One B run on level 5 was invalidated twice by infrastructure failures and counted on its third launch. B and C thus reach verdicts at the same rate. In the seven C runs with completed final proofs, the submitted bound is a kernel-checked theorem under explicit model premises; B supplies its own executable solver. The statement and cross-representation audits in the Lean verification package differ from the Python certificate checks: vacuous premises can be reported even when their main theorems compile, while the tested reversed-bound and translation-drift mutants are rejected (Appendix C). This comparison concerns the tested verification packages, rather than a general inability of Python to perform such audits.

## C Frozen rules on unseen states, and Lean against Python

Both tests were preregistered, with criteria frozen before running, and their main readings were audited independently. Test data come from the official game sources run locally; replaying the recorded winning runs on these sources reproduces every recorded frame (lp85 200/200, ft09 168/168), and no file in the benchmark repository is named after, or generated for, the modified levels.

**Frozen rules on unseen states.** For lp85 and ft09 we compare the concepts written during play (before Structure & Simplify: 19 for lp85, 9 for ft09) with the rules after it (6 and 7), both frozen at the benchmark commit. Test transitions come from random button presses (seeds 0–9) and keep only states whose screen never occurs in the recorded run. Three sets per game: new trajectories on the original levels (T1), and two modified levels. For lp85 the modification swaps tiles on level 6 (same mechanics, new layout). For ft09 it changes level 2's hidden click pattern from the identity to a plus

shape, which the screen does not show. A rule set accepts a true transition if no rule judges it false; it rejects a wrong one, made by the benchmark's own five corruption operators, if some rule judges it false. Table 11 gives the readings; confidence intervals resample whole episodes. The modified levels come in pairs. In lp85 tile swap U, one goal tile is moved so that level 6 cannot be won; D is U with one more swap, which makes it winnable again. In ft09 pattern change FU, the plus-shaped pattern makes level 2 unwinnable; FD adds a one-pixel change to that level's template, which makes it winnable again. All four were checked by exhaustive search on the modified engine.

Table 11: Frozen rules on unseen transitions: before / after Structure & Simplify (%). U, D: the unwinnable and winnable tile swaps of lp85 level 6; FU, FD: the unwinnable and winnable pattern changes of ft09 level 2 (see text). "Wrong rejected" is the range over the four corruption operators for which both rule sets can read both frames; the fifth, which edits changing cells, is reported in the text.

| Set | $n$ | True accepted | Wrong rejected | Next frame predicted |
|---|---|---|---|---|
| lp85 T1 (new trajectories) | 2,073 | 100 / 100 | 99.5–100 / 99.5–100 | 99.95 / 99.95 |
| lp85 tile swap U | 2,080 | 100 / 100 | 100 / 100 | 100 / 100 |
| lp85 tile swap D | 2,080 | 100 / 100 | 100 / 100 | 100 / 100 |
| ft09 T1 (new trajectories) | 2,026 | 99.0 / 100 | 99.0–100 / 100 | — |
| ft09 pattern change FU | 2,015 | 0 / 0 | — | — |
| ft09 pattern change FD | 2,016 | 0 / 0 | — | — |

The preregistered criterion for a benefit of Structure & Simplify (a gain of at least 5 points) cannot be met because both rule sets are near ceiling; we claim no benefit here. On the ft09 pattern change both rule sets reject every true transition, all through the rule that nothing else changes: each click now changes 110 or 146 cells instead of 38 on the original level. The models fail loudly on dynamics they could not have observed, rather than accepting them silently. Next-frame prediction for lp85 compares the abstracted permutation with the union of the earlier concepts' predictions on tile cells; a first version also required the earlier concepts to predict the step bar, which the abstracted rule does not predict either, and was discarded as unequal.

**Lean against Python on the same model.** We compare three routes to the same impossibility bound on three models: the reference models of the two sealed `sc25` levels, which match the engine exactly (2,675 and 3,502 states), and the merged `lp85` model (869 states). V1 is a Python search that reports the least winning budget; V2 is a Python checker of the certificate conditions; V3 is the Lean development, checked by the kernel. The Python and Lean versions of each model agree edge by edge, and on the unmodified models all three routes give the true bounds (19, 36 and 16). Each model receives 21 mutants, frozen with a hash before the runs: errors in the certificate generator (G), errors in the model definition (M1–M6), a change to only one of the Python and Lean versions (M7), and errors in the statement (S1–S7: unsatisfiable premises or an empty set of start states, a wrong goal set, a reversed bound, a statement that refers to another version of the model). Table 12 gives the readings for the three models together.

Table 12: Mutation test of three routes to the same bound, three models together. n/a: the route does not read the object that the mutant changes.

| Mutant | V1 Python search | V2 Python checker | V3 Lean |
|---|---|---|---|
| generator error: invalid certificate rejected | n/a | 12/12 | 12/12 |
| of which the conclusion is false on the engine | n/a | 9/9 | 9/9 |
| model error (M1–M6) rejected | none | none | none |
| Python and Lean versions differ (M7) rejected | 0/3 | 0/3 | 3/3 |
| vacuous premises or empty start set (S1, S2) reported | 0/6 | 0/6 | 6/6 |
| reversed bound (S4) rejected | n/a | 0/3 | 3/3 |
| wrong goal set, other model version (S3, S7) rejected | none | none | none |

On errors in the certificate generator, Lean and the Python checker reject exactly the same certificates, including three whose conclusion is true but whose certificate is invalid. The Lean verification package additionally reports vacuous premises in all six tested cases, although their main theorems

still compile, and rejects the three reversed-bound and three cross-representation mutants. A kernel checks a proof against its stated premises; it does not by itself reject an unsatisfiable premise. The scope of these statement checks differs from that of the Python certificate checker. No route checks whether the model itself is right: mutants of the model definition pass all three, including those whose conclusion is false on the engine. That is the job of the comparison with the engine's record (Appendix B), and the rules a Lean conclusion depends on can be read off the proof: for example, the dependencies of one `sc25` model mutant lose the rule for eating fuel, and those of several `lp85` mutants switch to another set of buttons.

## D Additional figures

Figure 4 is the complete model of `lp85`, whose panels (d) and (e) are part (b) and (c) of Figure 3.

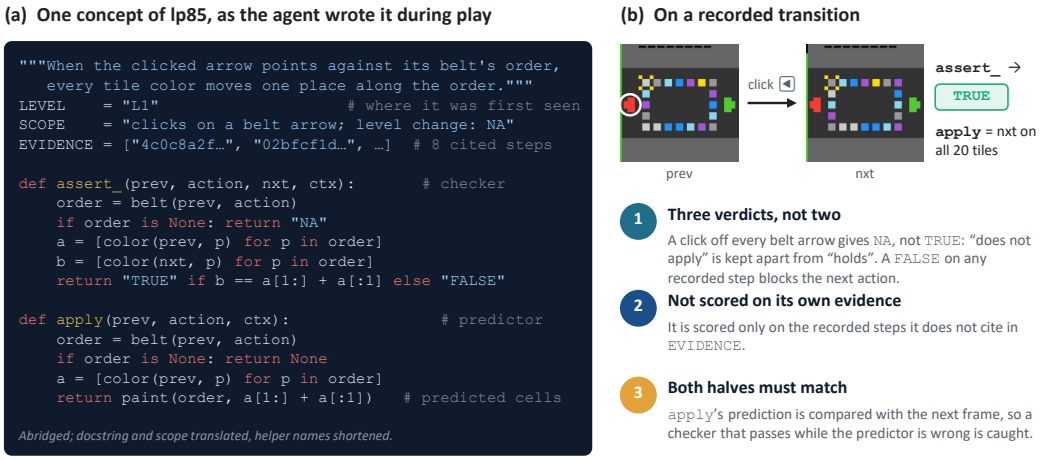



Figure 1: A concept. (a) The concept of `lp85` for a click on an arrow that points against its belt's order, abridged from the file the agent wrote during play. (b) On the first click of the certified run of level 1, its checker returns `TRUE` and its predictor reproduces the next frame on all 20 tiles. A concept is scored only on recorded transitions outside its own `EVIDENCE`, and a `FALSE` on any recorded transition, or an exception counted as ERR, blocks the next action (Eq. (1)).

## E Additional tables

Table 13: Performance on the 25 public ARC-AGI-3 games. Standard-harness agents have no code execution; the other systems execute code. Action totals refer to certified clears, excluding preceding uncertified exploration. Baseline results are from (ARC Prize Foundation, 2026; Rodionov, 2026a; Zhao et al., 2026); per-game results are in the supplementary material.

| System | Code | Score | Games at 100 | Certified actions | Actions/ human |
|---|---|---|---|---|---|
| Claude Opus 5 | no | 40.7 | 5 | — | — |
| GPT-6 Astra | no | 68.3 | 10 | — | — |
| GPT-6 Astra (adapter) | yes | 100.0 | 25 | — | — |
| baseline1 | yes | 99.0 | 23 | 8,347 | 0.49 |
| Memento 3 | yes | 100.0 | 25 | 7,518 | 0.44 |
| **AWM (ours)** | yes | **99.9** | 23 | **7,448** | **0.43** |

Official scores and certified actions (Table 13), then per-game official scores and per-game certified actions.

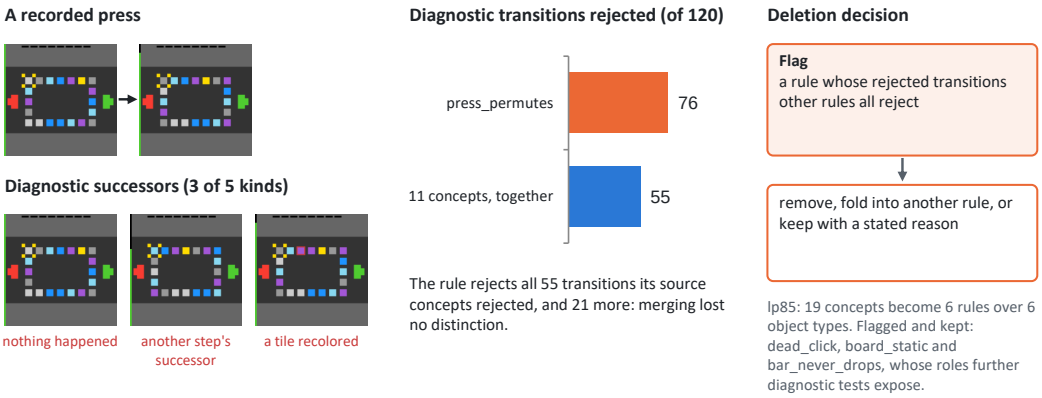



Figure 2: Diagnostic transitions, the merge check and deletion in `lp85`. Diagnostic transitions are built from recorded ones (three of the five constructions are shown on a real press). On the 120 diagnostic transitions $\mathcal{N}$ of this game, the merged rule `press_permutes` rejects all 55 that its eleven source concepts rejected together, and 21 more, so the merge lost no tested distinction; these counts are for the eleven merged concepts and their rule, while the whole book and the final six rules reject 117 and 119 (Table 3). A rule whose rejections the others cover is flagged and then removed, folded into another rule, or kept with a stated reason; the three flagged rules of `lp85`, `dead_click`, `board_static` and `bar_never_drops`, are kept because further diagnostic tests expose roles absent from this set.

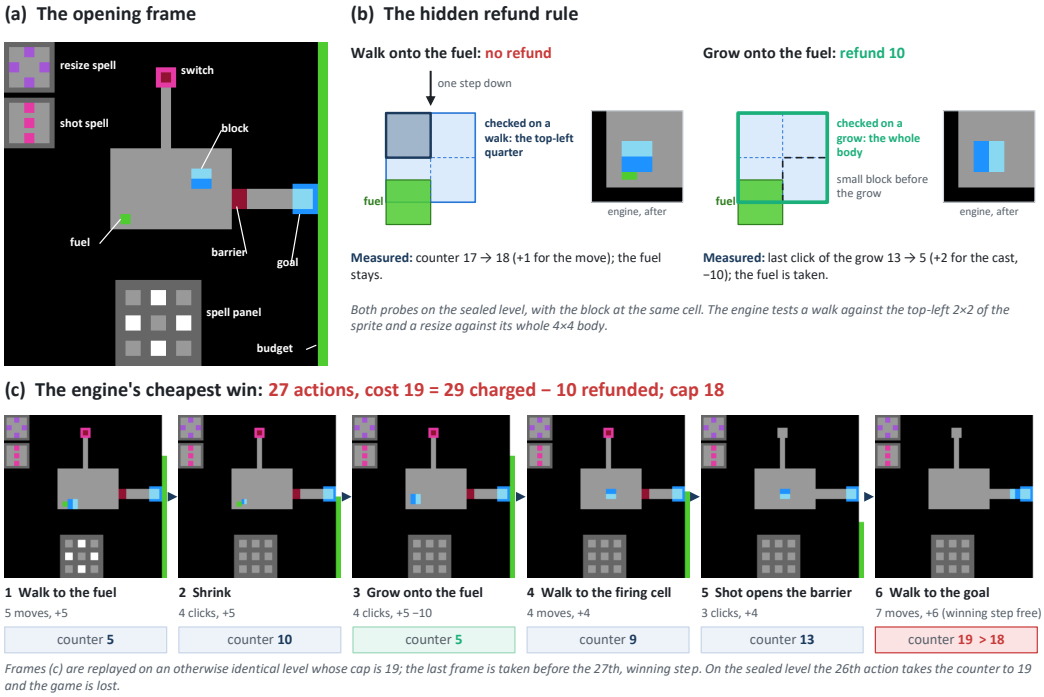



Figure 3: Sealed level 4 of `sc25`. (a) The opening frame. (b) The hidden refund rule: with the fuel under the bottom-left quarter of a large block, walking there pays nothing, because the engine checks only the top-left quarter, while growing there refunds 10; this is the rule the framework run that proved 20 misread. (c) The engine's cheapest win, 27 actions at a cost of 19 against a cap of 18, with the counter after each stage. Frames are rendered from the real engine with the official palette; those in (c) are replayed on an otherwise identical level whose cap is 19, since on the sealed level the 26th action ends the game.

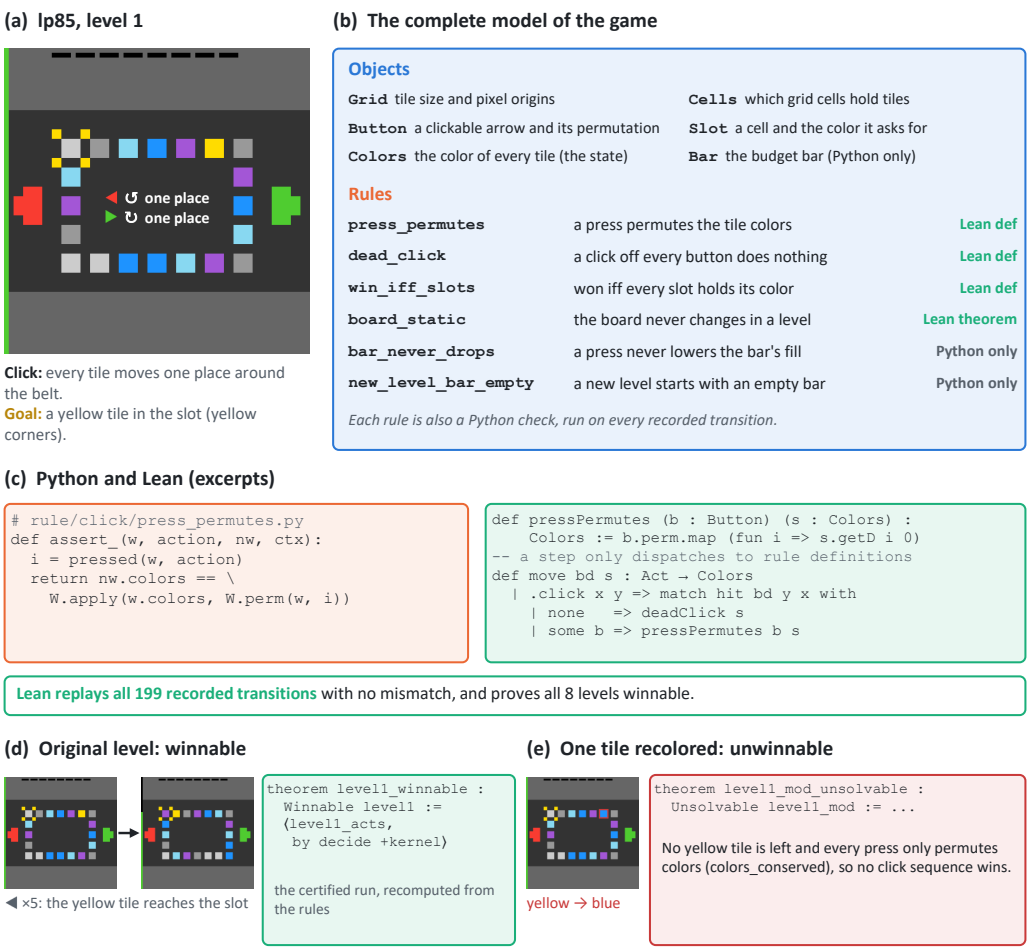



Figure 4: The complete algebraic world model of an ARC-AGI-3 game. (a) Level 1 of `lp85`: 20 colored tiles form a closed belt, and each click on an arrow moves every tile one place around the belt, counterclockwise for the left arrow and clockwise for the right; the level is won when a yellow tile stands in the slot marked by yellow corners. (b) All objects and rules of the model, with the form each takes in Lean; the two rules about the budget bar stay in Python. (c) Excerpts: one rule as its Python check, the same rule as a Lean definition, and the Lean step, which only dispatches to rule definitions. Lean replays every recorded transition of the game with no mismatch. (d) On the original level, Lean checks a winning witness, the certified run. (e) With the only yellow tile recolored, the conservation law of `press_permutes` shows that no click sequence wins. Both statements are theorems about the model.

Table 14: Official scores on the 25 public ARC-AGI-3 games. Baselines are the official ARC Prize results: Claude Opus 5 (High) and GPT-6 Astra (Max) under the standard harness, which provides no code execution, and GPT-6 Astra (Max) under its provider-adapter harness. AWM is scored on its certified run. baseline1 from its ARC Prize scorecard; Memento 3 as reported in (Zhao et al., 2026).

| | | Official score (max 100) | | | | | |
|---|---|---|---|---|---|---|---|
| Game | Lv. | Claude Opus 5 | GPT-6 Astra | GPT-6 Astra (adapter) | baseline1 (Rodionov, 2026b) | Memento 3 (Zhao et al., 2026) | **AWM (ours)** |
| ar25 | 8 | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 | **100.0** |
| bp35 | 9 | 0.0 | 2.2 | 100.0 | 100.0 | 100.0 | **100.0** |
| cd82 | 6 | 0.6 | 82.9 | 100.0 | 100.0 | 100.0 | **100.0** |
| cn04 | 6 | 47.6 | 100.0 | 100.0 | 100.0 | 100.0 | **100.0** |
| dc22 | 6 | 44.8 | 100.0 | 100.0 | 100.0 | 100.0 | **100.0** |
| ft09 | 6 | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 | **100.0** |
| g50t | 7 | 0.0 | 0.0 | 100.0 | 100.0 | 100.0 | **100.0** |
| ka59 | 7 | 10.4 | 75.0 | 100.0 | 100.0 | 100.0 | **97.6** |
| lf52 | 10 | 1.8 | 10.9 | 100.0 | 100.0 | 100.0 | **100.0** |
| lp85 | 8 | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 | **100.0** |
| ls20 | 7 | 5.4 | 83.5 | 100.0 | 100.0 | 100.0 | **100.0** |
| m0r0 | 6 | 28.6 | 71.4 | 100.0 | 100.0 | 100.0 | **100.0** |
| r11l | 6 | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 | **100.0** |
| re86 | 8 | 58.3 | 77.8 | 100.0 | 100.0 | 100.0 | **100.0** |
| s5i5 | 8 | 100.0 | 100.0 | 100.0 | 100.0 | 100.0 | **99.3** |
| sb26 | 8 | 77.8 | 100.0 | 100.0 | 100.0 | 100.0 | **100.0** |
| sc25 | 6 | 0.0 | 47.6 | 100.0 | 84.2 | 100.0 | **100.0** |
| sk48 | 8 | 8.3 | 19.7 | 100.0 | 100.0 | 100.0 | **100.0** |
| sp80 | 6 | 56.3 | 100.0 | 100.0 | 100.0 | 100.0 | **100.0** |
| su15 | 9 | 10.4 | 27.4 | 100.0 | 100.0 | 100.0 | **100.0** |
| tn36 | 7 | 6.0 | 53.6 | 100.0 | 90.1 | 100.0 | **100.0** |
| tr87 | 6 | 47.6 | 100.0 | 100.0 | 100.0 | 100.0 | **100.0** |
| tu93 | 9 | 3.6 | 3.1 | 100.0 | 100.0 | 100.0 | **100.0** |
| vc33 | 7 | 98.8 | 73.3 | 100.0 | 100.0 | 100.0 | **100.0** |
| wa30 | 9 | 10.6 | 80.0 | 100.0 | 100.0 | 100.0 | **100.0** |
| Mean | 183 | 40.7 | 68.3 | 100.0 | 99.0 | 100.0 | **99.9** |
| Games at 100 | | 5 | 10 | 25 | 23 | 25 | **23** |

Table 15: Certified actions per game. Human, baseline1 and Memento 3 as reported in (Zhao et al., 2026); AWM from its certified runs (Table 2). AWM's counts cover only its certified sessions, which in 22 games follow uncertified sessions of the same run, so its totals are not compared with the other systems'. *AWM total*: every action AWM sent to the engine in the run, including uncertified sessions, failed attempts and resets (the opening reset is not counted); 3.3 times its certified actions overall, 1.0–10.0 times per game.

| Game | Human | baseline1 | Memento 3 | AWM | AWM / Human | AWM total |
|---|---|---|---|---|---|---|
| ar25 | 748 | 271 | 253 | 253 | 0.34 | 253 |
| bp35 | 651 | 532 | 429 | 374 | 0.57 | 374 |
| cd82 | 171 | 93 | 82 | 70 | 0.41 | 661 |
| cn04 | 789 | 201 | 201 | 261 | 0.33 | 745 |
| dc22 | 1,228 | 898 | 643 | 1,037 | 0.84 | 2,664 |
| ft09 | 208 | 85 | 75 | 80 | 0.38 | 168 |
| g50t | 879 | 363 | 307 | 288 | 0.33 | 811 |
| ka59 | 730 | 362 | 341 | 421 | 0.58 | 4,061 |
| lf52 | 1,339 | 767 | 894 | 905 | 0.68 | 905 |
| lp85 | 388 | 90 | 95 | 79 | 0.20 | 200 |
| ls20 | 776 | 465 | 473 | 368 | 0.47 | 1,467 |
| m0r0 | 1,107 | 215 | 220 | 215 | 0.19 | 486 |
| r11l | 233 | 97 | 75 | 81 | 0.35 | 184 |
| re86 | 1,255 | 559 | 625 | 547 | 0.44 | 2,153 |
| s5i5 | 638 | 283 | 261 | 461 | 0.72 | 2,618 |
| sb26 | 213 | 140 | 124 | 124 | 0.58 | 424 |
| sc25 | 350 | 619 | 173 | 146 | 0.42 | 401 |
| sk48 | 1,070 | 419 | 407 | 263 | 0.25 | 1,407 |
| sp80 | 518 | 193 | 144 | 138 | 0.27 | 371 |
| su15 | 361 | 117 | 104 | 90 | 0.25 | 904 |
| tn36 | 317 | 292 | 133 | 125 | 0.39 | 292 |
| tr87 | 414 | 147 | 148 | 119 | 0.29 | 430 |
| tu93 | 462 | 201 | 208 | 187 | 0.40 | 566 |
| vc33 | 447 | 293 | 204 | 167 | 0.37 | 538 |
| wa30 | 1,843 | 645 | 899 | 649 | 0.35 | 1,521 |
| Total | 17,135 | 8,347 | 7,518 | 7,448 | 0.43 | 24,604 |

## References


Diego Aineto and Enrico Scala. Action model learning with guarantees. In *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pp. 801–811, 2024. doi: 10.24963/kr.2024/75.

ARC Prize Foundation. GPT-6 Astra: ARC-AGI results. `https://arcprize.org/results/openai-gpt-6-astra`, 2026. Public Demo, 25 environments; accessed 2 October 2026.

Edmund M. Clarke, Orna Grumberg, and David E. Long. Model checking and abstraction. *ACM Transactions on Programming Languages and Systems*, 16(5):1512–1542, 1994. doi: 10.1145/186025.186051.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. doi: 10.1016/S0004-3702(98)00023-X.

Sergey Rodionov. Executable world models for ARC-AGI-3 in the era of coding agents. In *Artificial General Intelligence: 19th International Conference, AGI 2026*, Lecture Notes in Computer Science. Springer, 2026a. arXiv:2605.05138.

Sergey Rodionov. Do coding agents need executable world models, simplification, and verification to solve ARC-AGI-3? *arXiv preprint arXiv:2607.15439*, 2026b.

Haoyu Zhao, Zhengxu Yu, Zhiyuan He, Meng Fang, Rasul Tutunov, Haitham Bou-Ammar, Weilin Luo, and Jun Wang. Memento 3: Model-based recursive self-improvement through reflective rulebooks. *arXiv preprint arXiv:2610.11794*, 2026.